

TDA / Algor³
1^{er} C 202
2^{do} Parcial

Nombre y Apellido	L.U.	N° Orden	Materia y turno

Duración: 5 horas. Este examen es individual y a libro cerrado.

Los ejercicios 1 a 4 valen 5 puntos si se responden correctamente, -2 puntos si se responden incorrectamente. Cada uno tiene una única opción correcta. Marcar más de una opción se considera una respuesta incorrecta.

Los ejercicios 5 y 6 valen 10 puntos si se responden correctamente.

Los ejercicios 7 a 10 valen 15 puntos si se responden correctamente.

La nota de aprobación es de 60/100 puntos

Ejercicio 1: Un coloreo en un grafo es una asignación de colores a todos sus vértices de manera tal que si dos vértices son adyacentes entonces no reciben el mismo color. Un grafo de n vértices que es un ciclo es coloreable con 2 colores

☐ si y solo si n es potencia de 2 ☒ si y solo si n es par ☐ si y solo si $n \geq 2$ ☐ siempre

Ejercicio 2:Cuál es la complejidad temporal de sort en función de la longitud n del array de entrada?

```
def sort_rec(arr, i, j):
    if j < i+2:
        return
    if j == i+2:
        if arr[i+1] < arr[i]:
            swap(arr[i+1], arr[i])
        return
    n = j-i
    sort_rec(arr, i, j-n//3)
    sort_rec(arr, i+n//3, j)
    sort_rec(arr, i, j-n//3)
```

```
def sort(arr):
    sort_rec(arr, 0, len(arr))
```

☐ $\Theta(n \log_{3/2}(n))$ ☐ $\Theta(n^{\log_{3/2}(3)})$ ☐ $\Theta(n^{3/2 \cdot \log_3(n)})$ ☐ $\Theta(n^3)$

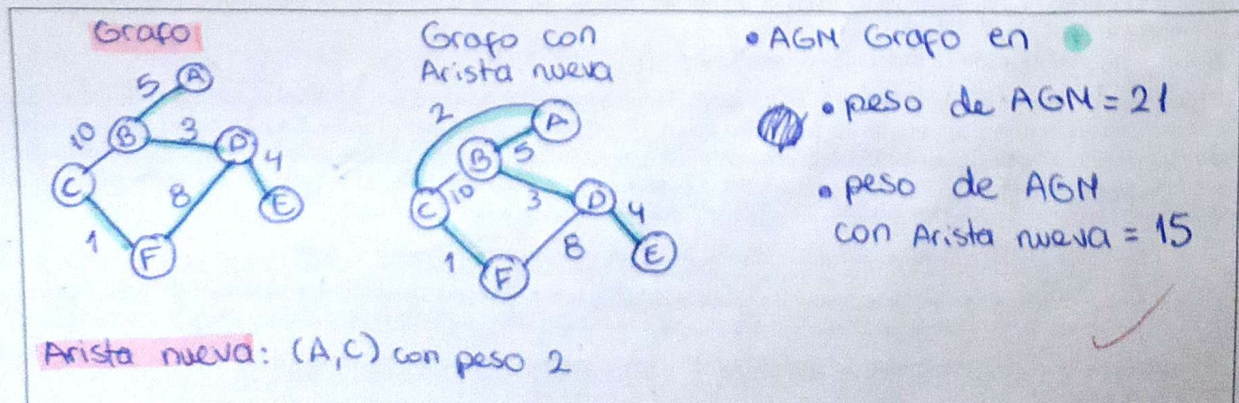
Ejercicio 3: Al aplicar el algoritmo de BFS en un grafo grilla (donde cada nodo es una celda, y está conectado con sus vecinos, que son 4 para las celdas internas, 2 para las esquinas y 3 para los bordes que no son esquinas). Si se ejecuta una búsqueda BFS comenzando en esquina (0,0) puede asegurarse que un vértice (i,j) se visita después que el vértice (2,3) cuando

☐ $i > 2$ ☐ $j > 3$ ☒ $i + j > 5$ ☐ nunca puede asegurarse, depende del recorrido

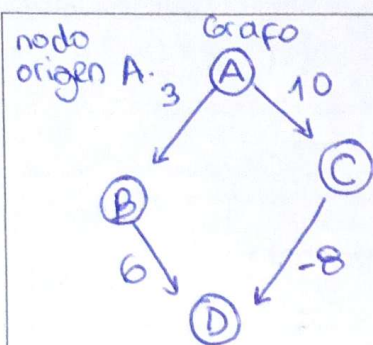
Ejercicio 4: Si una función recursiva tiene complejidad $T(n) = bT(n/b) + \Theta(n)$ con $b > 1$, entonces $T(n)$ es

☐ $\Theta(n)$ ☒ $\Theta(n \log n)$ ☐ $\Theta(n \log^2 n)$ ☐ $\Theta(n^2)$

Ejercicio 5: Dar un ejemplo de un grafo conexo pesado y una arista nueva, tal que agregarle la arista al grafo reduzca el peso del AGM del grafo.



Ejercicio 6: Sabemos que el algoritmo de Dijkstra puede fallar si en un grafo dirigido pesado se tienen pesos negativos. Dar un ejemplo de una instancia (grafo y nodo de origen) donde esto ocurra.



Si corro Dijkstra en este grafo desde A, el algoritmo falla (no me devuelve las longitudes de los caminos mínimos de A a todos los nodos). Esto se ve con el nodo D, ~~cuya~~ ya que $d(A,D)=2$ y Dijkstra va a devolver $d(A,D)=9$.

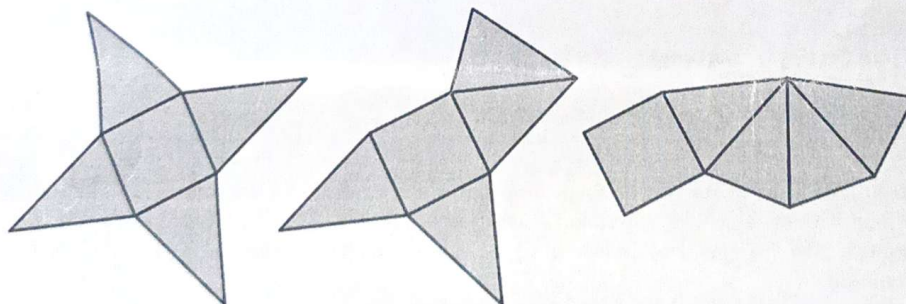
esto es lo que debería devolver.

Ejercicio 7: Sea $I = \{[l_1, r_1], [l_2, r_2], \dots, [l_n, r_n]\}$ un conjunto de intervalos sobre la recta real. Queremos cubrir todos los intervalos colocando la menor cantidad de puntos $x_1, \dots, x_k \in \mathbb{R}$, tales que para todo $[l_i, r_i]$ exista algún $x_j \in [l_i, r_i]$. Consideramos el siguiente algoritmo:

- Ordenar los intervalos por su extremo derecho r_i en orden creciente.
- Inicializar el conjunto de puntos $S = \emptyset$
- Para cada intervalo $[l_i, r_i]$, si no existe ningún punto de S que lo cubra, agregamos r_i a S .

Demostrar que este algoritmo devuelve una solución óptima, es decir, que S tiene la mínima cantidad posible de puntos que cubren todos los intervalos.

Ejercicio 8: Querés armar una pirámide 3D. Para eso, primero tenés que imprimir y recortar un papel, y luego doblar y pegar los bordes expuestos hasta formar la figura 3D. Los lados de la base mide 4cm y las otras aristas, 6cm.



En la ilustración se ven tres ejemplos de cómo podrías armar el recorte. Tenés que encontrar, de todas las formas posibles de armarlo, una que requiera pegar la menor longitud total de bordes para armar la figura 3D. Modelar con el problema de árbol generador mínimo. Esto es, definir un grafo tal que resolver el problema propuesto sea equivalente a encontrar un árbol generador mínimo en ese grafo y **explicar** como recuperar la solución al problema original a partir del AGM del grafo.

Ejercicio 9: Se tiene un digrafo G con conjunto de nodos $\{1, \dots, n\}$. La *belleza* de una permutación f es la cantidad de índices $i \in \{1, \dots, n\}$ tales que $(i, f(i))$ es un eje del digrafo. Modelar el problema de encontrar la permutación más bella usando max-flow.

Nota: una permutación es una función biyectiva $f : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$.

Ejercicio 10: James Bo está atrapado en un grafo de n vértices y m aristas ($m > n$). Para escapar debe ir del vértice en el que se encuentra a alguno de los marcados como "de salida" en a lo sumo t turnos. En cada turno puede recorrer exactamente una arista o permanecer en el nodo en el que está. Hay un problema: en cada turno $1, \dots, t$, puede haber algunas aristas con trampas mortales activadas. El servicio de inteligencia oriental le proveyó con una servilleta que contiene:

- La matriz M de tamaño $m \times t$ que indica si la arista i puede atravesarse en el turno j .
- La estructura del grafo, donde además están señalados tanto el vértice donde él se encuentra prisionero como el conjunto S de vértices de salida.

Dar un algoritmo de complejidad temporal $O(mt)$ para encontrar un camino de escape de la mínima cantidad de turnos, si existiera.

②

② LAKS

hago referencia al orden por fin(r_i) de [l_i, r_i].

Ej 7) Supongamos que O' es una solución óptima, y S es la solución del algoritmo greedy. Qvq: O y S tienen la misma cantidad de puntos (la mínima). Para esto haré inducción para ver que siempre puedo extender una solución parcial de S a una solución óptima O .

Ambos ~~algoritmos~~ ^{soluciones} son conjuntos (sin orden), pero para demostrar voy a usar una lista que tenga los elementos del conjunto con orden creciente (esto no afecta las soluciones, ya que tengo los mismos puntos).

Entonces llamo O y G a las listas ordenadas de la solución óptima y greedy respectivamente.

Caso base: $i=1$ (mi solución parcial tiene 1 elemento).

Tengo la solución $O = (O_1, O_2, \dots, O_k)$. ~~El elemento de~~ la solución parcial greedy ~~cuando~~ agrega el primer elemento y por como funciona el algoritmo, se que ~~agrego el r_i~~ ~~el más chico~~ más chico de todos los intervalos $\Rightarrow r_i \leq r_j \forall j, 1 \leq j \leq n$; ~~entonces~~ además, en O debo abarcar todos los intervalos (porque es óptimo) $\Rightarrow O_1 \leq r_i \Rightarrow$ los puedo intercambiar (ambos pertenecen al primer intervalo y si O_1 pertenecía a un intervalo con fin superior $\Rightarrow r_i$ estará contenido también en ese intervalo). Entonces puedo extender a $(r_i, O_2, \dots, O_k) = (G_1, O_2, \dots, O_k)$.

H.I: Puedo extender a una solución óptima si mi solución parcial tiene hasta i elementos.

$G = (G_1, G_2, \dots, G_i)$ extiende a $O = (G_1, G_2, \dots, G_i, O_{i+1}, \dots, O_k)$

Qvq: Al agregar el punto G_{i+1} en el algoritmo greedy, igual lo puedo extender a una solución óptima. (se puede extender a óptimo con $i+1$ elementos).

siguiente, me refiero al
que tenga el siguiente menor r_i .
↑ PERO ESE
PUEDE ESTAR
CUBIERTO
POR G_i

yo se que tanto O_{i+1} como G_{i+1} abarcan el siguiente intervalo ~~que~~ al que abarca G_i (ya que están ordenados los puntos y en una solución óptima no voy a tener más de 1 punto que abarque el mismo intervalo).

Entonces, de nuevo, este intervalo empieza luego del fin del intervalo de G_i .

Entonces de nuevo tengo que, por como funciona el algoritmo greedy, $G_{i+1} = r_b$ (donde r_b es el fin del intervalo siguiente al que abarca G_i) $\Rightarrow$

$O_{i+1} \leq G_{i+1}$ (porque si no, O_{i+1} no abarcaría ese intervalo) ~~no~~ y entonces G_{i+1} abarca cualquier intervalo siguiente que podría abarcar $O_{i+1} \Rightarrow$ los puedo intercambiar ~~no~~: $O = (G_1, G_2, \dots, G_i, G_{i+1}, \dots, O_k)$ entonces puedo extenderlo a una solución óptima.

Por como funciona la inducción, me queda que puedo seguir extendiendo a una solución óptima hasta que termine el algoritmo greedy $\Rightarrow$ greedy devuelve una solución óptima $\Rightarrow S$ tiene la mínima cantidad posible de puntos que cubren todos los intervalos.

PARA DECIR QUE ES ÓPTIMA FALTA JUSTIFICACIÓN
¿CÓMO SABES QUE NO FALTAN PUNTOS?

NUNCA EXPLICAS POR QUÉ SOLO CON QUE
DOS PUNTOS ESTÉN EN EL MISMO

INTERVALO SABES QUE CUBREN
LO MISMO. QUE ES CENTRAL
A LA DEMO

R

Ej 10) • Puedo ir a lo sumo en t turnos.

• En cada turno recorre una arista o permanece en la que está.

• Tengo matriz $m \times t$ que dice si puedo pasar por arista i en el turno j .

• Tengo los vértices de salida.

• Sé donde estoy.

turno t

~~Puedo hacer un BFS desde el vértice donde está~~

Puedo modelar un grafo ~~(arista, turno)~~ con nodos ~~(arista, turno)~~ ~~matriz~~ ~~gráfico~~ ~~turno~~. Luego conecto con

~~un nodo con otro~~ $\Leftrightarrow$ existe un nodo u con v

$\Leftrightarrow \exists (u, v)$ en G y ~~alternando~~ si tengo (u, t_u) y (v, t_v)
 $t_u \leq t_v - 1$ (si ~~un nodo~~ el nodo de salida pertenece a ~~al~~ turno anterior ~~al~~ ^{de} llegada).

Estos nodos (arista, turno) solo existen si puedo pasar por la arista en ese turno (si no puedo pasar en ese turno, no existe la tupla en mi modelo).

Corro BFS desde donde está James Bo y ~~salir~~ el primer nodo $\in S$ que aparezca, será al que deberá ir James en su camino de escape. ~~(siempre que~~
~~hay tupla)~~

También conecta ~~el~~ ~~nodo~~ v ~~nodo~~ u , ~~siempre~~

conecta (u, t_u) a $(u, t_u + 1)$ (Todo nodo se conecta con el mismo en el siguiente turno).

El grafo que modelé es dirigido.

La complejidad de BFS es ~~$O(V+E)$~~ $O(V+E)$

Tengo $V = mt$, $E =$

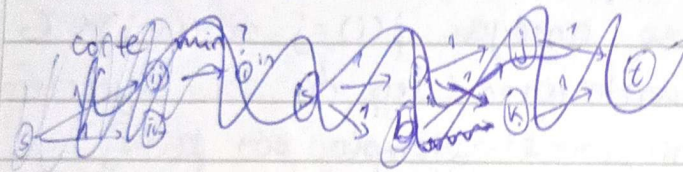
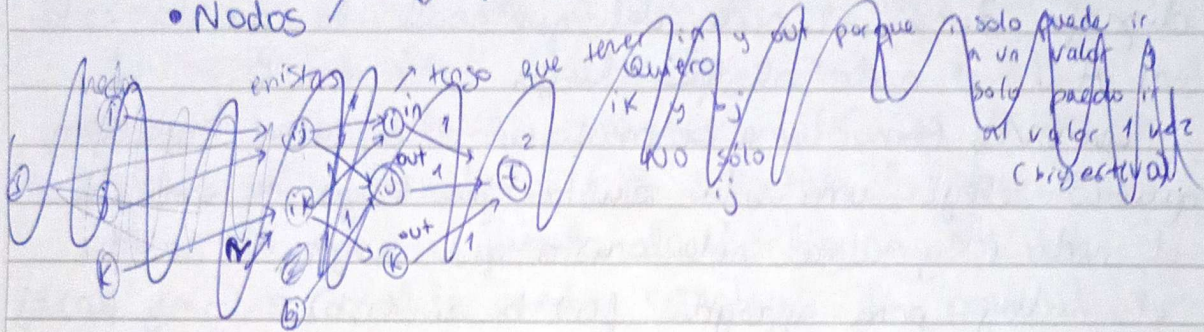
?

ver a
modo por
como lo
modelante
respuestas
con

③

B

Ej 9 • Aristas > Digrafo G
• Nodos

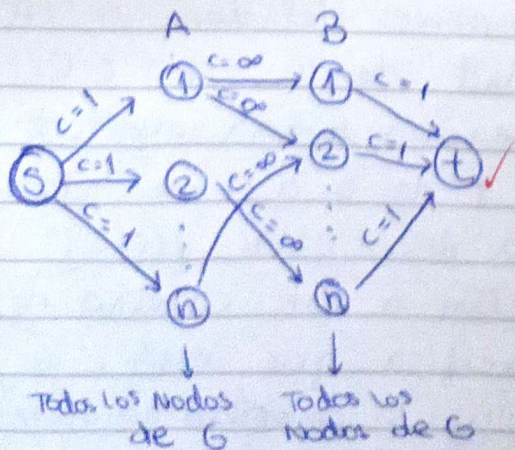


Mi red (V, E) es la siguiente:

- V = Nodos del digrafo G duplicados (por cada nodo del digrafo, hay 2 nodos que lo representan en la red).

Tengo una bipartición (A, B) donde A tiene todos los nodos de G sin repetidos, al igual que B). Además tengo el sumidero y fuente.

- E = La fuente conecta con los nodos de A con capacidad 1. ~~Para~~ $\forall (u, v) \in G$ tengo una arista del nodo u en la bipartición A que va al nodo v en B con capacidad ∞ . Los nodos de B conectan con el sumidero con capacidad 1.



Encontrar el flujo máximo en este modelo, resuelve el problema de la permutación mas bella.

Flujo máximo = Belleza de la permutación más linda.

Hay arista entre ellos si ~~hay~~ existe esa arista en G .

Explicación: Yo quiero que mi permutación tenga la mayor cantidad de aristas $(i, f(i)) \in G$.

En la red representamos todas las aristas del digrafo, donde el nodo del que sale la arista está en A y el nodo al que llega está en B.

~~Como una permutación es una biyección,~~ Como una permutación es una biyección, quiero elegir una sola arista que tenga de entrada el nodo i y una sola arista que tenga de salida el nodo j, para agregar $f(i)=b$ si $(i,b) \in G$ y $f(a)=j$ si $(a,j) \in G$, pero no agregar $f(i)=j$ si $(i,j) \in G$ si ya agregué las 2 anteriores (deja de ser biyección y visquiera sería función en el ejemplo con $f(i)=j$).

Para esto las capacidades que salen de S significan

→ con cierta permutación

"este nodo $\in \text{Dom}(f)$ " y por eso tienen capacidad 1.

Lo mismo para con los nodos que van a t, ~~que~~ ^{que} significan ~~que el nodo es uno~~ ^{que} cuyas aristas significan "este nodo es parte de la imagen" y tienen capacidad 1 para que la función sea biyectiva.

La capacidad de las aristas del medio ~~son~~ es indiferente porque ya tengo las restricciones que necesito. Entonces ~~el flujo máximo es~~ la configuración del flujo máximo (el flujo de cada arista) representa ~~comparar los~~ ^{que} ~~valores de~~ ^{que} ~~mi dominio y van~~ ^{que} a donde debo enviar ~~los~~ ^{ciertos} elementos del dominio (A) para tener la máxima cantidad de $(i, f(i)) \in G$.

~~Un flujo 1 desde S a i, y luego algo~~

~~Cada camino de flujo 1 de S a t me indica~~ Cada camino de flujo 1 de S a t me indica que en la permutación más bella debe estar $f(i)=j$

si $i \in A$, $j \in B$ y ambos pertenecen a este camino de flujo 1. Si hay flujo 0, significa que esos i' y j' que pertenecen al camino de flujo 0 ~~no~~ no tienen restricción en la permutación mas bella, por lo que ~~no~~ tengo

$f(i')=a$, siendo a cualquier número de 1 a n que no sea $f(b)=a$ para $b \neq i'$

Es decir, si alguien ~~va~~ ~~ya~~ ya iba a "a",
i' no puede ir a ~~ella~~ "a" (porque es una función).

~~Por mismo~~ ~~pasan~~ ~~ya~~ es análogo para j', que
puede ser resultado de cualquier $f(K)$ si K no va
a ningún otro resultado (Es biyectiva la función).

De esta forma, viendo la configuración del
flujo en mi modelo puedo construir la permutación
más bella.

Todo lo que dije de que la configuración
del flujo me da ciertas permutaciones válidas
es cierto gracias a la conservación de flujo y $\text{flujo} \leq \text{cap}$
(elementos del $\text{Dom}(f)$ van a un sólo lugar por cap ~~1~~ ~~3~~ de la fuente)
(elementos de la $\text{Im}(f)$ son resultado de un único $f(i)$ por
 cap 1 al sumidero)

Para toda arista el flujo
no supera su capacidad.

B

